

PowerPy

A physics-first framework for solar-array power & thermal analysis

This document is what you take into the room. The left margin tells you **what to say**; the green notes tell you **why a design choice was made**; the code blocks are the exact lines you can point at. The last two pages are a rehearsed **Q&A** so nothing catches you off guard.

What PowerPy does, in one breath

It takes a single Excel workbook (`params.xlsx`) plus a couple of JSON catalogues describing a solar cell and a panel substrate, and produces an **auditable engineering report** answering two coupled questions:

ELECTRICAL

How much power does the array deliver — at beginning of life and after years of radiation damage, across every mission phase and bus voltage? Built bottom-up: cell → string → section → panel → array.

THERMAL

How hot does each cell get, and what happens when one fails? A 2-node radiative balance per tile, with optional in-plane heat spreading, plus worst-case and Monte-Carlo failure studies.

The one-sentence thesis to open with. “PowerPy replaces a pile of ad-hoc spreadsheets and one-off scripts with a single typed pipeline: one source of truth in, validated physics in the middle, a reproducible PDF out — and every number on the page can be traced back to a line of code or a cell in the workbook.”

Contents

1. The 90-second pitch
2. Architecture & data flow
3. Single source of truth: `params.xlsx` → typed schemas
4. The electrical model (single-diode + curve algebra)
5. Radiation & temperature: how a cell ages
6. The thermal solver (the heart of the framework)
7. Coupling electrical → thermal
8. The layout/palette idea & the recent cell-reference change
9. Failure studies: worst-case & Monte-Carlo
10. Report generation: HTML heat-map & LaTeX PDF
11. Why it's trustworthy: the 65.26 °C oracle & tests
12. Anticipated questions & answers

1 · The 90-second pitch

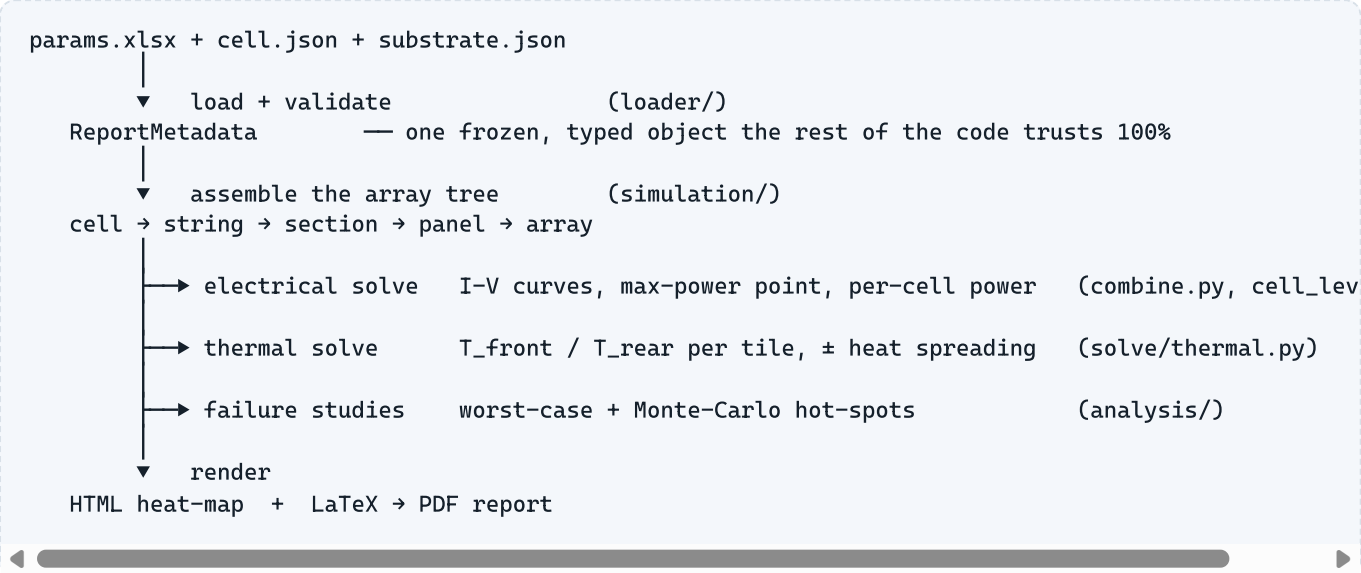
SAY THIS FIRST

“A solar array on a spacecraft has to meet a power requirement not on day one, but after 15 years of radiation in orbit, in the worst thermal case, even with a failed cell. Proving that used to mean stitching together spreadsheets and a legacy script. PowerPy does the whole chain in one tool — and it’s built so every step is independently testable and every output is reproducible from version-controlled inputs.”

The three claims worth defending

Claim	What backs it
One source of truth	Every input lives once in <code>params.xlsx</code> or a referenced JSON. The workbook itself is regenerated from <code>build_params.py</code> , so it diffs cleanly in git — no hidden cells, no copy-paste drift.
Physics you can read	The solvers are a few hundred lines of NumPy. The single-diode fit and the 2-node thermal balance are written exactly as the equations — no black box.
Reproducible & validated	A single sunlit cell converges to 65.26 °C , a hand-worked result that is pinned by a test. Any change to the solver that breaks that number fails CI.

The shape of the pipeline (memorise this line)



Why a pipeline and not one big script? Each box is a package that imports nothing it doesn’t need and can be unit-tested alone. The thermal solver doesn’t know what Excel is; the loader doesn’t know what radiation is. That separation is what lets us say “this number is correct” about any single stage.

2 · Architecture & data flow

Eight packages, each with one job. Point at this table when asked “how is the code organised?”

Package	Responsibility
<code>config/</code>	Physical descriptions: the substrate (optics + conduction) and the panel layout (grid + palette).
<code>schemas/</code>	Immutable, validated dataclasses — the typed vocabulary (<code>ReportMetadata</code> , <code>CellElectrical</code> , <code>CircuitLayout</code> ...). The single source of truth in memory.
<code>loader/</code>	The Excel → schema boundary. Each sheet has one loader; bad data fails <i>here</i> , loudly, not deep in a solve.
<code>simulation/</code>	Builds the array tree, resolves the per-phase environment, runs the electrical evaluation.
<code>solve/</code>	The thermal solver (2-node, independent or laterally coupled) and the electro-thermal coupling.
<code>analysis/</code>	Engineering studies: equilibrium temperatures, worst-case search, Monte-Carlo failure sampling.
<code>render/</code>	Jinja → LaTeX → PDF; matplotlib figures.
<code>reporting/</code>	The standalone thermal HTML heat-map + JSON (the <code>powerpy run</code> path).

The data model that ties it together

Everything downstream consumes one object — `ReportMetadata` — composed by seven loaders. This is the “trust boundary”: once it is built, no further validation is needed.

```
# loader/report.py – the whole input side, in one function
def load_report_data(params_file, data_dir) → ReportMetadata:
    return ReportMetadata(
        document      = load_document_metadata(params_file, data_dir),
        cell          = load_cell_parameters(params_file, data_dir),    # + referenced JSON
        mission       = load_mission_parameters(params_file),
        array_layout   = load_array_layout(params_file),
        losses        = load_losses(params_file),
        radiation_fluxes = load_radiation_fluxes(params_file),
        structure      = load_report_structure(params_file),
    )
```

Why frozen dataclasses? Inputs never mutate mid-run. If `ReportMetadata` is immutable, a bug can’t silently change a cell parameter halfway through a sweep. Every derived quantity is a *pure function* of these inputs — which is exactly what makes results reproducible and the Monte-Carlo seed-stable.

IF ASKED “WHY REBUILD EXCEL FROM A PYTHON SCRIPT?”

“Because a binary `.xlsx` is invisible to git. `build_params.py` defines every sheet, row and unit in code, so a parameter change shows up as a readable diff and can be reviewed like any other code.”

3 · Single source of truth: the workbook → typed schemas

The workbook holds **mission & design inputs**; the JSON catalogues hold **reusable physics** (a cell type, a substrate). The split matters: the same cell JSON is shared by many missions.

Two sheet shapes, one parser each

Shape	Used for	Example sheets
Key-value	One value per parameter	document , cell_params , requirement
Long-format	A value <i>per phase / per launch config / per level</i>	mission_param , losses , radiation_fluxes , sections

WHAT THE CELL CATALOGUE CARRIES

The cell JSON (e.g. `data/cells/3G30LARS_GEO.json`) is loaded and mapped into a typed `CellElectrical` . Note the loader accepts both modern and legacy key names — so old project files still load.

```
# loader/cell.py - tolerant mapping (modern OR legacy keys)
def pick(*keys, default=None):
    for k in keys:
        if k in data: return data[k]
    return default

return CellElectrical(
    isc_bol = float(pick("isc_bol", "isc")),      voc_bol = float(pick("voc_bol", "voc")),
    imp_bol = float(pick("imp_bol", "imp")),      vmp_bol = float(pick("vmp_bol", "vmp")),
    temp_coeff_isc = float(pick("temp_coeff_isc", default=0.0)),
    temp_coeff_voc = float(pick("temp_coeff_voc", default=0.0)),
    degradation = data.get("degradation", {}), regressors = regressors,
    alpha = float(pick("alpha", default=0.0)), # the cell's solar absorptivity
    epsilon = float(pick("epsilon", default=0.0)),
    area_m2 = float(pick("area_m2", "area", default=0.0)),
)
```

Why `pick()` instead of a strict schema? The real legacy project files use `isc / voc` ; our newer convention uses `isc_bol / voc_bol` . One tolerant reader means we never have to hand-migrate historical datasets to demo or validate against them — they Just Work, and the typed object downstream is identical either way.

WHAT GOES IN THE WORKBOOK (LONG-FORMAT EXAMPLE: RADIATION)

Radiation dose is given *per launch config × per phase × per parameter* (current axis vs voltage axis), because a GEO transfer orbit and the operational orbit accumulate very different fluences.

launch_config	phase	param	value [e/cm²]
dual	End_of_Life	isc	1.0 × 10 ¹⁵
dual	End_of_Life	voc	5.0 × 10 ¹⁴

POINT TO MAKE

"The current axis and the voltage axis degrade *differently* under radiation, so we track two separate doses. That's why you'll see `dose_i` and `dose_v` threaded through the whole electrical model."

4 · The electrical model — single-diode + curve algebra

A solar cell is modelled with the classic **single-diode** equivalent circuit. The terminal relation is implicit in the current because of the series-resistance drop $I \cdot R_s$:

$$I = I_{ph} - I_0 \cdot (e^{(V + I \cdot R_s)/V_t} - 1) - (V + I \cdot R_s) / R_{sh}$$

single-diode terminal equation

WHY WE DON'T NEED SPICE: A CLOSED-FORM FIT FROM 4 DATASHEET POINTS

Instead of fitting numerically, we recover the five parameters analytically from the four points every datasheet gives us — I_{sc} , I_{mp} , V_{mp} , V_{oc} — taking the shunt branch as effectively open (R-series assumption, $R_{sh} \approx 10^9 \Omega$). This is the legacy `RSeriesModel` algebra, re-derived:

```
# simulation/cell_level.py - fit_rseries()
vt = (vmp * (isc - imp)) / imp
i0 = isc / (np.exp(voc / vt) - 1.0)
rs = (vt / imp) * np.log((isc - imp) / i0) - (vmp / imp)
return DiodeParams(iph=isc, i0=i0, vt=vt, rs=max(rs, 0.0), rsh=1e9, voc=voc)
```

$$V_t = V_{mp}(I_{sc} - I_{mp}) / I_{mp} \cdot I_0 = I_{sc} / (e^{V_{oc}/V_t} - 1) \cdot R_s = (V_t / I_{mp}) \cdot \ln((I_{sc} - I_{mp}) / I_0) - V_{mp} / I_{mp}$$

Why closed-form, not a numerical fit or ngspice? Three reasons: (1) it's *deterministic* — no solver tolerance to argue about; (2) it needs no external binary, so the tool runs in a locked-down environment; (3) it exactly reproduces the legacy method, so results are directly comparable to the spreadsheets it replaces. An ngspice path still exists and transparently falls back to this.

DRAWING THE CURVE: ONE VECTORISED NEWTON LOOP

To get the full I-V curve we sweep V and solve the implicit equation for I at every voltage at once — Newton on a whole array, converging in a handful of steps:

```
# simulation/cell_level.py - single_diode_iv()
V = np.linspace(0.0, p.voc, n); I = np.full_like(V, p.iph)
for _ in range(80):
    vd = V + I * p.rs
    e = np.exp(np.clip(vd / p.vt, -700.0, 700.0)) # clip => no overflow
    f = p.iph - p.i0 * (e - 1.0) - vd / p.rsh - I # residual
    df = -p.i0 * e * (p.rs / p.vt) - p.rs / p.rsh - 1.0 # d f / d I
    I = I - f / df # Newton step
    if np.max(np.abs(f / df)) < 1e-12: break
```

FROM ONE CELL TO THE WHOLE ARRAY: TWO RULES, APPLIED RECURSIVELY

Every join in the tree is either series or parallel. The entire array I-V is these two functions composed:

SERIES (CELLS IN A STRING)

Same current; **voltages add**. Limited by the smallest I_{sc} .

```
isc_min = min(isc of each child)
i_grid  = linspace(0, isc_min, n)
v_total =  $\sum$  interp V(I) of each child
# a weak cell caps the string current
```

PARALLEL (STRINGS, SECTIONS, PANELS)

Same voltage; **currents add**. Limited by the smallest V_{oc} .

```
voc_min = min(voc of each child)
v_grid  = linspace(0, voc_min, n)
i_total =  $\sum$  interp I(V) of each child
```

THE CLEAN LINE TO DELIVER

"The array model is just these two rules — series adds voltage, parallel adds current — applied at four levels. A string is series cells, a section is parallel strings, a panel is parallel sections, the array is parallel panels. No special cases."

5 · Radiation & temperature — how a cell ages

The four operating points aren't fixed: they shift with **temperature** and degrade with accumulated **radiation dose**. The model applies three multiplicative gains to the begin-of-life values:

```
# simulation/cell_level.py - operating_points()
dT = env.temperature_c - env.reference_temperature_c
f_temp_i = 1.0 + (e.temp_coeff_isc / 100.0) * dT      # temperature, current axis
f_temp_v = 1.0 + (e.temp_coeff_voc / 100.0) * dT      # temperature, voltage axis
f_rad_i = _remaining_factor(e.degradation["isc"], env.dose_i) # radiation, current
f_rad_v = _remaining_factor(e.degradation["voc"], env.dose_v) # radiation, voltage

gain_i = f_temp_i * f_rad_i * env.current_loss      # × harness/optical losses
gain_v = f_temp_v * f_rad_v * env.voltage_loss

isc = e.isc_bol * gain_i;   imp = e.imp_bol * gain_i
voc = e.voc_bol * gain_v;   vmp = e.vmp_bol * gain_v
```

$$I_{sc} = I_{sc}^{BOL} \cdot (1 + \alpha_I \Delta T) \cdot RF_I(\Phi_I) \cdot L_I \quad V_{oc} = V_{oc}^{BOL} \cdot (1 + \alpha_V \Delta T) \cdot RF_V(\Phi_V) \cdot L_V \quad BOL \rightarrow \text{operating point}$$

THE REMAINING-FACTOR CURVE: INTERPOLATE IN LOG-FLUENCE

Radiation damage is logarithmic in fluence — each decade of dose costs a roughly constant fraction. So the degradation table is interpolated in `log10(fluence)`, not linearly:

```
# _remaining_factor() - Φ in units of 1e14 e-/cm^2
fluence = dose_1e14 * 1e14
if not curve or fluence ≤ 0.0: return 1.0      # begin-of-life = no damage
xs = np.log10([max(p[0], 1.0) for p in pts])    # log-fluence knots
ys = [p[1] for p in pts]                       # remaining factors
return np.interp(np.log10(fluence), xs, ys)
```

Why split dose into a current axis and a voltage axis? Displacement damage hits the photocurrent and the junction voltage by different amounts. Using one “dose” for both would over- or under-predict end-of-life power. The cell JSON ships separate `r_isc / r_imp` and `r_voc / r_vmp` curves precisely so we can honour that asymmetry.

Why $\pm\Delta T$ about a reference temperature? Datasheet points are quoted at a reference (here `t_ref = 28 °C`). The temperature coefficients are the measured slopes about that point — so $(1 + \alpha \cdot \Delta T)$ is a first-order, physically grounded correction, not a guess.

THE HEADLINE FOR THIS SLIDE

“This is where ‘end of life’ becomes a number. Feed in the EOL dose and the hot-case temperature, and the same four points come out shrunk — that shrunk curve is what has to still meet the power requirement.”

6 · The thermal solver — the heart of the framework

Each tile is a **two-node** object: a front face (sees the sun and deep space) and a rear face (sees albedo, planet IR, deep space), joined through the substrate by a conduction term. We solve the steady state where, at each node, **power in = power out**.

$$\begin{aligned} \text{front: } & \alpha_F A \cdot q_{\text{sun}} - \epsilon_F A \sigma (T_F^4 - T_{\text{sp}}^4) - P_{\text{elec}} + C_{\text{cond}}(T_R - T_F) = 0 \\ \text{rear: } & q_R - \epsilon_R A \sigma (T_R^4 - T_{\text{sp}}^4) - C_{\text{cond}}(T_R - T_F) - g_{\text{lat}} \cdot \Sigma (T_R - T_{\text{neighbour}}) = 0 \end{aligned} \quad \text{2-node steady-state balance}$$

where $C_{\text{cond}} = c_{\text{cond}} \cdot A$ (substrate through-thickness conduction) and the radiative loss is the Stefan–Boltzmann $\epsilon A \sigma T^4$ term. P_{elec} is the electrical power leaving the front node.

ONE ENTRY POINT, ONE SWITCH: g_{lat}

This is the single most important design decision in the framework, so it's worth dwelling on.

$g_{\text{lat}} = 0$	$g_{\text{lat}} > 0$
Tiles are thermally independent. Each tile's 2×2 Jacobian is inverted in closed form , vectorised over the whole array — the fast path.	Neighbouring tiles share heat in-plane. We assemble one sparse ($2N \times 2N$) Jacobian and take Newton steps.

Why build it as one solver with a switch, not two? Because $g_{\text{lat}} = 0$ is a *strict special case* of the coupled model — turning lateral conduction on only *adds* heat-spreading terms. So the coupled path is a superset: it can always reproduce the independent answer exactly. That gives us a free, built-in regression check, and it means we never maintain two physics models that could drift apart.

THE FAST PATH: CLOSED-FORM 2×2 , VECTORISED (NO PYTHON LOOP OVER TILES)

```
# solve/thermal.py - the g_lat == 0 Newton update, all tiles at once
f1 = qF - eF*A*SIGMA*(T1**4 - tsp4) - Pe + CA*(T2 - T1) # front residual
f2 = qR - eR*A*SIGMA*(T2**4 - tsp4) - CA*(T2 - T1) # rear residual

a = -4.0*eF*A*SIGMA*T1**3 - CA;    b = CA # 2x2 Jacobian per tile
c = CA;    d = -4.0*eR*A*SIGMA*T2**3 - CA
det = a*d - b*c
dT1 = -(d*f1 - b*f2) / det # closed-form inverse
dT2 = -(-c*f1 + a*f2) / det
```

THE COUPLED PATH: PRECOMPUTE THE CONSTANT STRUCTURE ONCE

Across Newton iterations only the radiative diagonal ($-4\epsilon A \sigma T^3$) changes. The conduction and the lateral coupling (a graph Laplacian of the tile-adjacency) are **fixed** — so we build them once and only add the changing diagonal each step. The in-plane heat flow is then a single sparse mat-vec, no per-neighbour loop:

```
# built ONCE: Laplacian L of the 4-neighbour grid + the constant Jacobian block
Lap = diags(deg) - adj
Jconst = bmat([[-CAAd - g_lat*Lap,  CAAd],
               [ CAAd, -CAAd - g_lat*Lap]])
# each iteration: flux is one Laplacian matvec; J = Jconst + changing radiative diagonal
f1 -= g_lat * Lap.dot(T1);  J = Jconst + diags(rad);  dx = spsolve(J, -f)
```

THE SENTENCE THAT LANDS THIS SLIDE

“Turn the one knob `g_lat` from zero up, and you go from ‘each cell alone’ to ‘cells sharing heat with their neighbours’ — same code, same equations, just more physics. A failed cell now warms the ones around it, which is exactly the effect we need to catch.”

7 · Coupling electrical → thermal

The two solvers meet at one term: `P_elec`, the electrical power leaving each cell. Its sign carries the entire physical story.

$P_{ELEC} > 0$ — A HEALTHY CELL

Energy leaves the tile as electricity. That power does **not** become heat → the cell runs **cooler**. It equals the extracted max-power point, $P = I_{mp} \cdot V_{mp}$.

$P_{ELEC} < 0$ — A FAILED CELL

A reverse-biased / failed-open cell **dissipates** the string current pushed through it. That energy stays as heat → a **hot spot**. Modelled as a negative `P_elec` on that tile.

```
# analysis/thermal_report.py — power extracted by one healthy cell
def _p_elec_per_cell(md, env) → float:
    cell = CellModel(md.cell); cell.apply(env)
    isc, imp, vmp, voc = cell.operating_points()
    return float(imp * vmp)          # the MPP cooling term [W]
```

```
# the solver zeroes power on tiles that can't generate (bare / diode)
Pe = np.where(props["generates_power"], Pe, 0.0)
```

Why is a hot spot a *negative* number and not a separate ‘failure mode’ flag? Because to the heat balance there is no difference between “a cell that stopped cooling” and “a cell that’s actively dumping watts” — both are just a power term at a node. Encoding failure as a value of `P_elec` means the *same* solver handles nominal and failed cases; the failure studies in §9 just vary that one array.

BARE REGIONS: FIRST-CLASS CITIZENS

A tile with no cell still absorbs sunlight but produces no power — so all of its absorbed sun becomes heat. That’s why a cut-out or keep-out region runs *hotter* than the cells around it and, with `g_lat > 0`, warms its neighbours. Capturing that was a core requirement, and it’s why the layout treats “bare” as a real tile type, not an absence.

WORTH SAYING OUT LOUD

“The coupling is deliberately one term. We don’t run a giant co-simulation — we compute the electrical operating point, hand its power to the thermal model as a per-tile number, and that’s the interface. It’s simple enough to audit and rich enough to show hot spots.”

8 · The layout/palette idea & the recent cell-reference change

A panel is described as a 2-D **grid of single-token tiles**, each key resolved through a **palette**. It reads like ASCII art, diffs cleanly, and — crucially — the grid *is* the thermal mesh: tile (r,c) borders (r±1,c) and (r,c±1), which is exactly the 4-neighbour adjacency the lateral solver needs.

```
{
  "pitch_mm": 40.0,
  "palette": {
    "A": {"is_cell": true, "cell_type": "3G30LARS_GEO", "string": "S1"},
    ".": {"is_cell": false}           // bare keep-out
  },
  "layout": ["A A . . A",
             "A A A A A"]
}
```

THE CHANGE MADE THIS SESSION NEW

Previously each cell palette entry *duplicated* the cell's optical numbers inline. Now a cell tile **references** its dataset by `cell_type`, and the loader pulls the cell's absorptivity straight from `data/cells/<cell_type>.json`. The rear faces and bare-tile optics come from the **substrate**:

```
# config/layout.py - from_dict(d, substrate=None)
if is_cell and cell_type and "alpha_front" not in clean:
    clean["alpha_front"] = _cell_front_alpha(cell_type) # the cell's alpha, from its file
if substrate is not None:                             # rear/bare optics belong to the board
    for fld in ("alpha_rear", "epsilon_front", "epsilon_rear"):
        clean.setdefault(fld, getattr(substrate, fld))
if not is_cell:
    clean.setdefault("alpha_front", substrate.alpha_front)
```

Why this matters (and why we did it now). It is the “single source of truth” principle applied to optics. A cell's absorptivity is a property of the *cell*; the front/rear emissivity of the board is a property of the *substrate*. Before, the same number was copy-pasted into every layout — change the cell and you had to hand-edit many files, with no guarantee they agreed. Now the layout says only *what sits where*; the physics is looked up. Explicit inline values still win, so every old layout keeps working unchanged.

From the grid, the layout hands the solver exactly three things — flat per-tile property arrays, the neighbour list, and which tiles generate power:

```
layout.prop_arrays() # {alpha_front, alpha_rear, epsilon_front, epsilon_rear, is_cell,
generates_power}
layout.neighbours()  # [(0,1),(0,2),(1,3), ...] the 4-neighbour adjacency = the heat mesh
```

IF HE PUSHES ON THE DESIGN

"The layout file is intentionally dumb — it's placement plus a reference. All the physics lives in the cell and substrate catalogues, which are shared and version-controlled. That's the whole point of the refactor."

9 · Failure studies — worst-case & Monte-Carlo

Once a failure is “just a per-tile power array”, exploring failures is cheap. Two complementary tools:

WORST-CASE SEARCH — A GREEDY HOT-SPOT HUNTER

Start with no failures; at each step, try failing each remaining cell, keep whichever drives the highest peak temperature, repeat up to `max_failures`. This deliberately finds **reinforcing clusters** — adjacent failures that, through lateral conduction, heat each other — which random sampling almost always misses.

AUTO-STOPPED MONTE-CARLO — SAMPLE UNTIL THE ANSWER IS PRECISE ENOUGH

Draw random failure patterns at a per-cell probability, solve each, and keep going until the **standard error** of the mean peak temperature drops below a target — then stop. No arbitrary “run 10,000 and hope.”

```
# analysis/study.py - the two public entry points (signatures)
worst_case_search(layout, max_failures=5, t_limit_c=150.0,
                  healthy_w=1.1, reverse_w=-9.6)          # greedy cluster
auto_monte_carlo(layout, p_fail=0.05, target_se=2.0,      # stop at SE ≤ 2 degC
                 batch=50, max_runs=2000, seed=0)
```

Why two methods instead of one? They answer different questions. Worst-case search asks “what is the *most damaging* thing that can happen?” — a deterministic bound for qualification. Monte-Carlo asks “what does a *realistic random* set of failures do, and how confident am I?” — a statistical expectation with an error bar. An engineer wants both: the bound and the distribution.

Why auto-stop on standard error? It makes the run self-justifying: you don’t pick the sample count, the *precision target* does. Report a mean peak of 168 °C with a ± 2 °C standard error and the number defends itself; a fixed 500-run number doesn’t.

The studies parallelise across threads — the sparse solves release the GIL — and the Monte-Carlo is seed-stable, so a reported figure can be regenerated bit-for-bit.

DEMO LINE IF YOU RUN IT LIVE

“`powerpy worst example_panel.json --max-failures 3` — watch it walk up the temperature, one cell at a time, to the worst three-cell cluster on the panel.”

10 · Report generation — two outputs, two purposes

A · THERMAL HTML HEAT-MAP

POWERPY RUN

A fast, dependency-free panel picture + a JSON “source of truth.” Each tile is coloured blue→amber→red by temperature; over-limit tiles are flagged. Pure Python string-building, no LaTeX — ideal for a quick look.

B · FULL ELECTRICAL PDF

POWERPY REPORT

The formal deliverable: Jinja templates → LaTeX → `pdflatex`, with matplotlib figures (I–V curves, loss budget, layout). The workbook’s `structure` sheet decides which sections appear and for which audience (customer vs engineer).

THE ELECTRICAL REPORT PIPELINE (END TO END)

```
build_electrical_report(params, out_pdf)
├─ load_report_data(params)           → ReportMetadata
├─ build the array (sections OR free-form circuit JSON)
├─ for each analysis case:
│   └─ env = environment_for_phase(report, phase, launch_config)
│       └─ res = run(array, env)         → Isc/Voc/Imp/Vmp/Pmp at every level
├─ Report.from_results(...).render(workdir) → report.tex + figures/*.png
└─ .compile_pdf(out_pdf)               → pdflatex x2 (so the TOC resolves)
```

WHY JINJA WITH LATEX-SAFE DELIMITERS

Default Jinja uses `{{ }}` — which collides with TeX’s own braces. The environment is reconfigured to use `<< >>` so templates stay valid LaTeX, with filters that escape special characters and format numbers (NaN-safe):

```
# render/templating.py
env = jinja2.Environment(
    block_start_string="<<%", block_end_string="%>>",
    variable_start_string="<<", variable_end_string=">>",
    undefined=jinja2.StrictUndefined,          # a missing field fails loudly
)
env.filters["tex"] = escape_tex                # \ & % $ # _ { } ~ ^
```

THE THERMAL HEAT-MAP COLOUR RULE

```
# reporting/report.py – temperature → RGB fill
x = clamp((t - tmin) / (tmax - tmin), 0, 1)      # normalise
r = 40 + 215*x;  g = 90 + 120*(1 - |2x-1|);  b = 200*(1-x) + 30
```

Why two report paths and not one? Speed vs formality. During design you iterate dozens of times — the HTML map renders instantly and needs no TeX toolchain. The PDF is the signed artefact you hand to a customer, so it goes through the full templated pipeline. Same physics underneath; different packaging for different moments.

IF PDFLATEX ISN'T INSTALLED ON HIS MACHINE

"The tool degrades gracefully — it still writes the complete `.tex` workspace and figures, it just skips the final compile. Nothing is lost."

11 · Why it's trustworthy — the oracle & the tests

The single strongest thing you can say about correctness: there is a **hand-worked answer** the code must match, and it's pinned by a test that runs on every change.

```
# the 65.26 degC oracle - a sunlit cell, no albedo/IR, no extracted power
s = load_substrate("msro_case2")
r = solve_thermal_for_substrate(s, area=0.003, p_sun=1367.0,
                               p_albedo=0.0, p_ir=0.0, p_elec=0.0)
assert abs(r.t_front_c[0] - 65.26) < 0.05      # matches the manual balance
```

Why one number carries so much weight. That single equilibrium temperature exercises the entire front/rear radiative balance, the conduction term, the Stefan–Boltzmann law and the substrate optics at once. If any of those were wrong, the number would move. Because `g_lat = 0` is a strict special case of the coupled solver, the *same* oracle also guards the lateral path — switch it on with zero conductance and you must get 65.26 °C back.

WHAT THE TEST SUITE COVERS

Test file	Guards
test_thermal_vectorized.py	The 65.26 °C oracle; substrate loading; the vectorised balance in isolation.
test_panel_thermal.py	Grid + palette parsing, adjacency, and that <code>g_lat=0</code> matches the independent oracle on a whole panel.
test_panel_study.py	Failure sweep, worst-case search, Monte-Carlo sampling.
test_electrical.py / test_circuit_layout.py	I–V curves, MPP, series/parallel combination, circuit topology parsing.
test_loader_validation.py / test_cli.py	Workbook → schema validation; the CLI commands end to end.

Current status: the full suite — **98 tests** — passes, including after this session's cell-reference refactor.

CLOSING LINE FOR THE TECHNICAL PART

"Every layer is testable alone, and the whole thing is anchored to a number we can compute by hand. That's the difference between 'the script gave 65 degrees' and 'the model is correct, and here's the proof that stays correct.'"

12 · Anticipated questions & how to answer

Ordered roughly by how likely they are. Each answer is short enough to say out loud, with the fact that backs it.

Q Why not just use SPICE / a commercial tool like a real circuit simulator?

A Three reasons. The closed-form single-diode fit is **deterministic** (no solver tolerance to defend), it needs **no external binary** so it runs in a locked-down environment, and it **exactly reproduces the legacy method** so results are directly comparable to the spreadsheets we're replacing. An ngspice path still exists for cross-checks and falls back to analytic automatically.

Q How do I know the thermal numbers are right?

A A single sunlit cell converges to **65.26 °C**, which we computed by hand from the radiative balance. That value is locked by a unit test, and because the coupled solver reduces to the independent one at `g_lat = 0`, the same check guards both code paths.

Q What exactly does "lateral conduction" add, and how do you set `g_lat` ?

A It lets neighbouring tiles exchange heat through the facesheet, so a hot cell warms its neighbours and a peak gets smeared out. `g_lat` comes from geometry — `g_lat = k · (edge · thickness) / pitch` — e.g. an aluminium facesheet ~0.045 W/K. Set it to zero and you recover the conservative "each cell alone" case.

Q Why is a failed cell modelled as negative power instead of a proper fault model?

A To the heat balance, a failed cell is just a node that dissipates watts instead of exporting them — so it's one term, `P_elec < 0`. That keeps a *single* validated solver for nominal and failed cases; the failure studies simply vary that per-tile array. The reverse-bias power itself comes from the string current the healthy cells push through the failed one.

Q Where do the radiation degradation curves come from, and why log-fluence?

A They're the manufacturer's measured remaining-factor data shipped in the cell JSON (separate curves for the current and voltage axes). Damage scales with the logarithm of fluence — each decade of dose costs a roughly constant fraction — so we interpolate in `log10(fluence)`, which is both physical and stable across the huge dose range.

Q Why split everything into so many packages? Isn't that over-engineered?

A Each package has one job and imports only what it needs, so it can be tested alone and a failure is localised. The thermal solver doesn't know Excel exists; the loader doesn't know radiation exists. That isolation is precisely what lets us make per-stage correctness claims — and it keeps `import powerpy` lightweight because heavy deps (scipy, pandas, LaTeX) load only when their layer runs.

Q What was the change you made today, and why?

A Layout files used to copy each cell's optical numbers inline. I made cell tiles **reference** their dataset by `cell_type`, so the absorptivity is read from `data/cells/<type>.json` and the rear/bare optics come from the substrate. It's the single-source-of-truth principle applied to optics: the layout now says only *what sits where*. Old layouts still work, and all 98 tests pass.

Q Can it handle a real, irregular array — not just a tidy rectangle?

A Yes. The grid+palette handles asymmetric panels, notches and bare keep-out regions directly, and for the wiring there's a free-form `CircuitLayout` (added recently) that decouples the electrical topology from the physical grid — arbitrary strings/sections/panels described in JSON.

Q How reproducible is a report? If I run it twice do I get the same PDF?

A Yes. Inputs are immutable, every derived quantity is a pure function of them, and the Monte-Carlo is seed-stable. Same workbook + same seed → same numbers, regenerable from version control.

Q What are the current limitations / what would you do next?

A It's steady-state thermal (a transient path exists but isn't the default), the coupling is one-directional electrical→thermal per case rather than a fully iterated electro-thermal loop, and the single-diode fit assumes an effectively-open shunt. None of these affect the qualification cases we target; they're the honest next steps if we need finer fidelity.

IF YOU GET A QUESTION YOU CAN'T ANSWER

Don't bluff. "Good question — that's in `<package>`; let me pull up the exact line rather than guess." The architecture is built so you *can* always point at the one place a behaviour lives.